```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
# Generate sample customer data
np.random.seed(42)

n = 2000

data = pd.DataFrame({
    "Age": np.random.randint(18, 70, n),
    "Gender": np.random.choice(["Male", "Female"], n),
    "Annual_Income": np.random.randint(15000, 150000, n),
    "Time_Spent_Minutes": np.random.randint(1, 120, n),
    "Pages_Viewed": np.random.randint(1, 30, n),
    "Previous_Purchases": np.random.randint(0, 20, n),
    "Cart_Items": np.random.randint(0, 10, n),
    "Discount_Used": np.random.choice([0, 1], n),
    "Device": np.random.choice(
        ["Mobile", "Desktop", "Tablet"], n
    )
})

# Generate purchase probabilities from customer behavior
score = (
    -2.0
    + 0.012 * data["Time_Spent_Minutes"]
    + 0.10 * data["Pages_Viewed"]
    + 0.12 * data["Previous_Purchases"]
    + 0.22 * data["Cart_Items"]
    + 0.45 * data["Discount_Used"]
    + 0.000005 * data["Annual_Income"]
)

probability = 1 / (1 + np.exp(-score))

data["Purchased"] = np.random.binomial(1, probability)

print("Dataset created successfully!")
display(data.head())
```

```
Dataset created successfully!
```

	Age	Gender	Annual_Income	Time_Spent_Minutes	Pages_Viewed	Previous_Purchases	Cart_Items	Discount_Used	Device	Purchase
0	56	Male	64812	110	22	11	0	0	Mobile	
1	69	Female	73643	11	25	16	9	0	Desktop	
2	46	Female	59900	92	28	14	7	0	Mobile	
3	32	Male	31288	54	5	8	8	1	Mobile	
4	60	Female	87692	105	24	6	6	0	Desktop	

```
print("Dataset shape:", data.shape)

print("\nDataset information:")
data.info()

print("\nMissing values:")
```

```
print(data.isnull().sum())

print("\nPurchase distribution:")
print(data["Purchased"].value_counts())

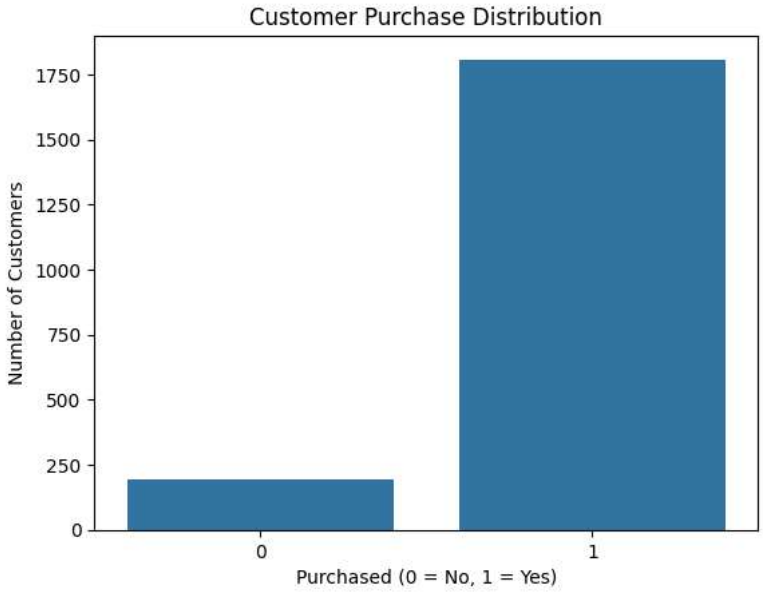
# Visualize purchase decisions
sns.countplot(data=data, x="Purchased")
plt.title("Customer Purchase Distribution")
plt.xlabel("Purchased (0 = No, 1 = Yes)")
plt.ylabel("Number of Customers")
plt.show()
```

```
Dataset shape: (2000, 10)

Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Age                 2000 non-null   int64
 1   Gender              2000 non-null   object
 2   Annual_Income       2000 non-null   int64
 3   Time_Spent_Minutes  2000 non-null   int64
 4   Pages_Viewed        2000 non-null   int64
 5   Previous_Purchases  2000 non-null   int64
 6   Cart_Items          2000 non-null   int64
 7   Discount_Used       2000 non-null   int64
 8   Device              2000 non-null   object
 9   Purchased           2000 non-null   int64
dtypes: int64(8), object(2)
memory usage: 156.4+ KB

Missing values:
Age                   0
Gender                0
Annual_Income         0
Time_Spent_Minutes    0
Pages_Viewed          0
Previous_Purchases    0
Cart_Items            0
Discount_Used         0
Device                0
Purchased             0
dtype: int64

Purchase distribution:
Purchased
1    1807
0     193
Name: count, dtype: int64
```




```
# Convert categorical columns into numerical columns
data_encoded = pd.get_dummies(
    data,
    columns=["Gender", "Device"],
```

```
    drop_first=False,
    dtype=int
)

# Separate input features and target variable
X = data_encoded.drop("Purchased", axis=1)
y = data_encoded["Purchased"]

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 1600
Testing samples: 400
```

```
# Create the Random Forest model
model = RandomForestClassifier(
    n_estimators=200,
    max_depth=12,
    random_state=42,
    class_weight="balanced"
)

# Train the model
model.fit(X_train, y_train)

print("Model trained successfully!")
```

```
Model trained successfully!
```

```
# Predict on test data
y_pred = model.predict(X_test)

# Calculate prediction probabilities
y_prob = model.predict_proba(X_test)[:, 1]

# Evaluate accuracy
accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy: {accuracy * 100:.2f}%")

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=["Not Purchased", "Purchased"],
    zero_division=0
))
```

```
Model Accuracy: 90.75%

Classification Report:
               precision    recall  f1-score   support

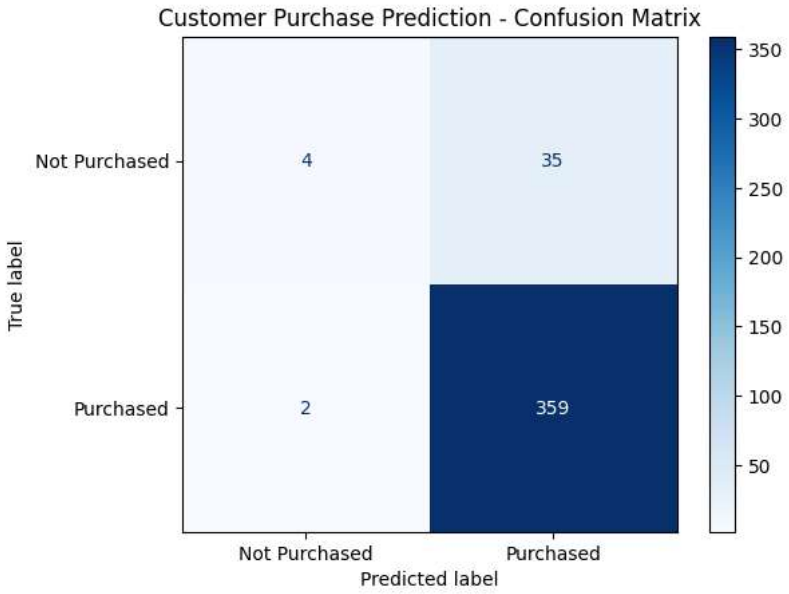
Not Purchased       0.67      0.10      0.18        39
    Purchased       0.91      0.99      0.95       361

     accuracy                           0.91       400
    macro avg       0.79      0.55      0.56       400
 weighted avg       0.89      0.91      0.88       400
```

```
cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Not Purchased", "Purchased"]
)
```

```
disp.plot(cmap="Blues")
plt.title("Customer Purchase Prediction - Confusion Matrix")
plt.show()
```

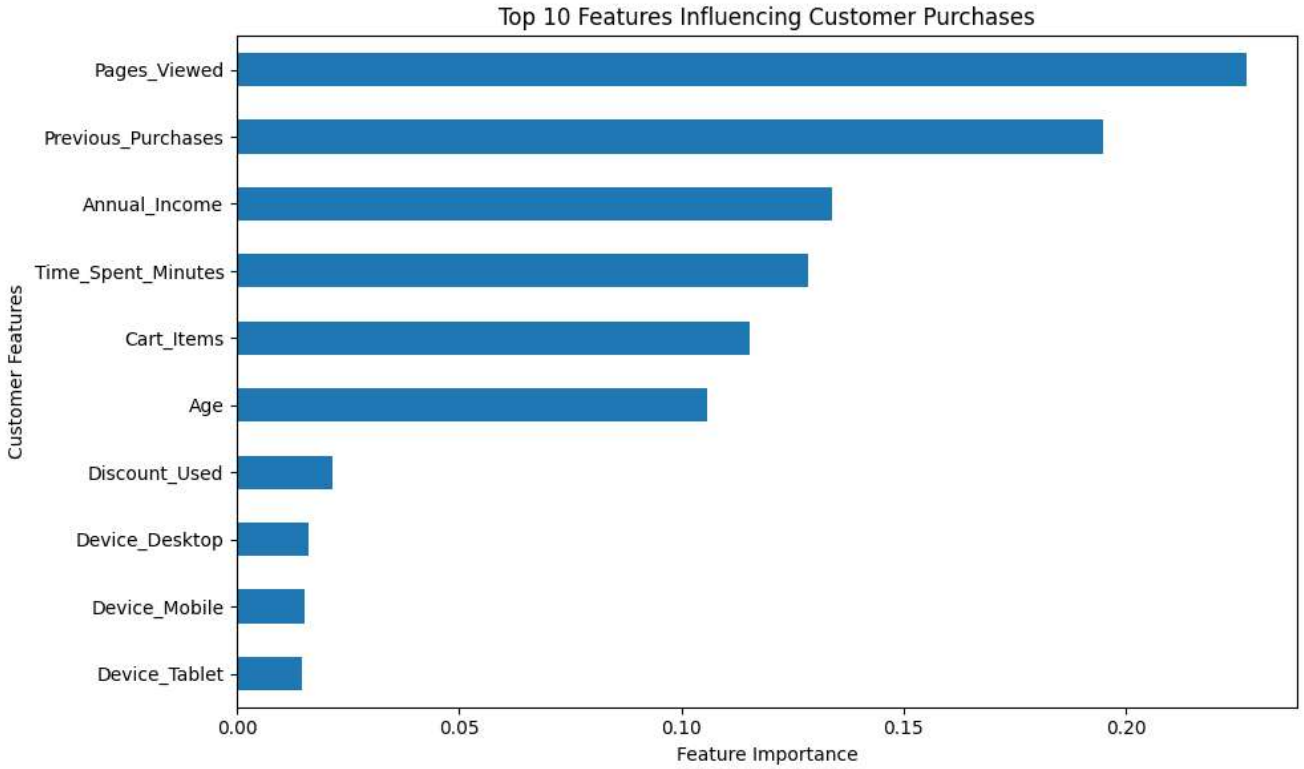



```
importance = pd.Series(
    model.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

print("Top factors influencing purchase decisions:")
print(importance.head(10))

plt.figure(figsize=(10, 6))
importance.head(10).sort_values().plot(kind="barh")
plt.title("Top 10 Features Influencing Customer Purchases")
plt.xlabel("Feature Importance")
plt.ylabel("Customer Features")
plt.tight_layout()
plt.show()
```

```
Top factors influencing purchase decisions:
Pages_Viewed          0.227140
Previous_Purchases    0.194894
Annual_Income         0.133776
Time_Spent_Minutes    0.128524
Cart_Items            0.115463
Age                   0.105842
Discount_Used         0.021419
Device_Desktop        0.016071
Device_Mobile         0.015159
Device_Tablet         0.014510
dtype: float64
```




```
# Example new customer
new_customer = pd.DataFrame([{
    "Age": 28,
    "Annual_Income": 65000,
    "Time_Spent_Minutes": 45,
    "Pages_Viewed": 12,
    "Previous_Purchases": 4,
    "Cart_Items": 3,
    "Discount_Used": 1,
    "Gender_Female": 1,
    "Gender_Male": 0,
    "Device_Desktop": 0,
    "Device_Mobile": 1,
    "Device_Tablet": 0
}])

# Ensure columns match the training data
new_customer = new_customer.reindex(
    columns=X.columns,
    fill_value=0
)

# Make prediction
prediction = model.predict(new_customer)[0]
purchase_probability = model.predict_proba(new_customer)[0][1]

print("Customer Purchase Prediction")
print("----------------------------")

if prediction == 1:
    print("Decision: Customer is likely to purchase.")
else:
    print("Decision: Customer is unlikely to purchase.")
```

```
print(f"Estimated purchase probability: "
      f"{purchase_probability * 100:.2f}%")
```

```
Customer Purchase Prediction
----------------------------
Decision: Customer is likely to purchase.
Estimated purchase probability: 66.34%
```